

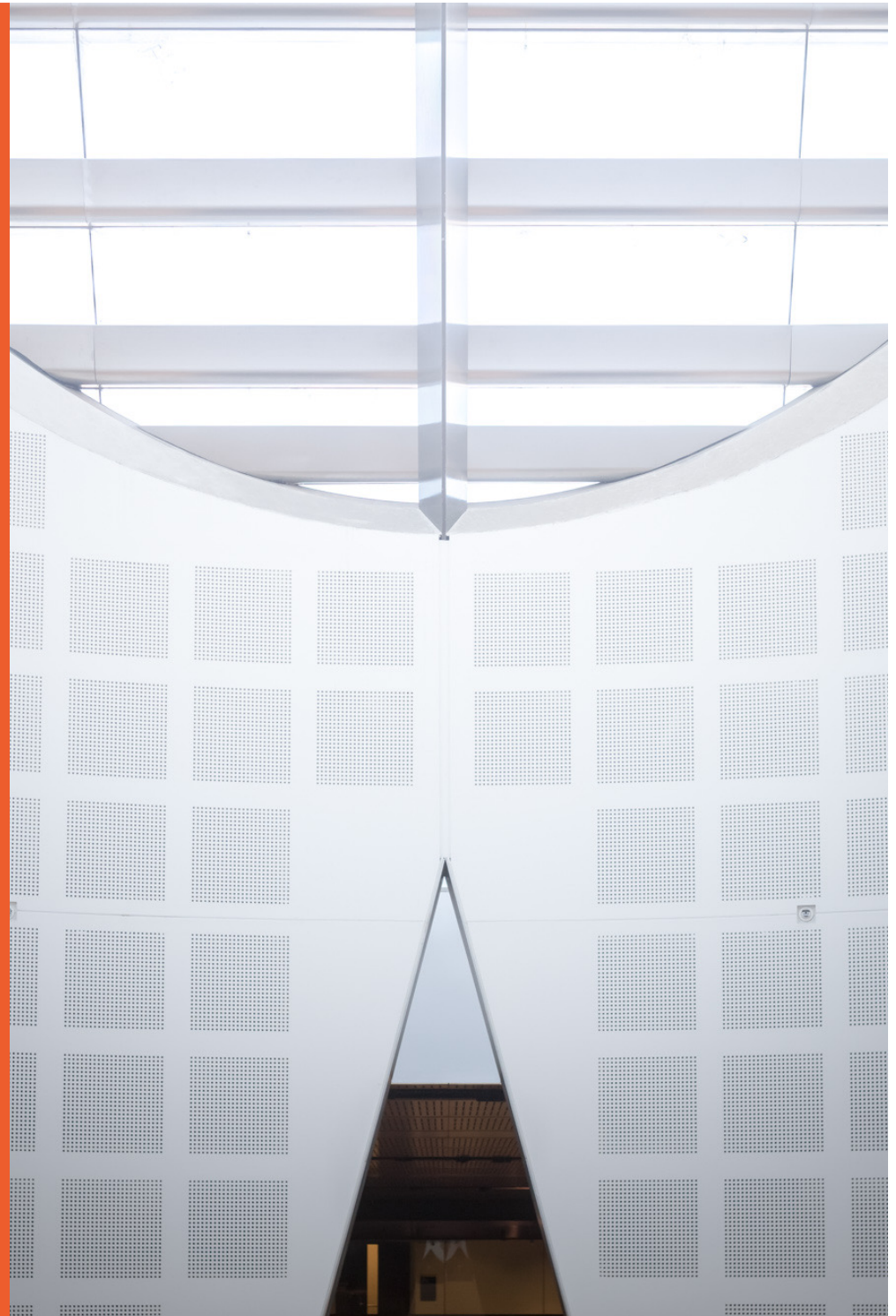
Quality and Testing

Presented by
Dr. Huaming Chen

ELEC3609/9609
Internet Software Platforms
2025 S2



THE UNIVERSITY OF
SYDNEY



We acknowledge the tradition of
custodianship and law of the Country on which
the University of Sydney campuses stand.
We pay our respects to those who have cared
and continue to care for Country.



THE UNIVERSITY OF
SYDNEY

Overview

- ✓ **Introduction**
- ✓ **Types of testing**
 - ✓ **Unit testing**
 - ✓ **Integration Testing**
 - ✓ **System Testing**
 - ✓ **Acceptance Testing**
 - ✓ **Regression Testing**
 - ✓ **Black-box/White-box Testing**
 - ✓ **Brute-forcing all input?**
 - ✓ **Fuzzing**
- ✓ **Testing Approach**
 - ✓ **Automated Testing**
 - ✓ **Manual Testing**
- ✓ **Linting**
- ✓ **Designing Test Cases**
 - ✓ **Equivalence Partitioning**
 - ✓ **Guideline Based**
 - ✓ **Examples**
 - ✓ **Odd/Even**
 - ✓ **Palindromes**
 - ✓ **Login Endpoint**
- ✓ **Code Coverage**
- ✓ **Writing Defensive Code**

Quality and Testing

Introduction

The quality of a web application is critical.

This lecture will not cover why testing is important or the value of testing. Almost every course you have taken will have covered this already.

This lecture will instead focus on fundamental testing concepts, like verifying the correctness of a web application and systems in general.

Applying a combination of these techniques for your assignment will assist in ensuring that your application has been deployed successfully and meets the requirements.

How to Test?

Test 101

Each test takes time to write, maintain and execute. As such, we should make sure each test provides value.

A good way to think about testing is that we have a budget for time spent on tests. Focus your time where it will provide the most value first.

Breadth vs depth vs critical functionality. E.g. Users can't do much unless they can log-in to the website.

Unit Testing

Unit, or functional, testing is the process of verifying functionality and components independently of the whole system.

The aim is to test the smallest testable piece of code that can be isolated, ideally a single function/method/subroutine.

One approach is to mock out other dependencies.

Example: testing the validate email function.

Integration Testing

Integration testing is the process of verifying the interactions/communication between software components.

A component is usually defined as a module or class. Stubs and drivers are commonly used when testing at this level.

System Testing

System testing is the process of verifying the behaviour and functionality of the entire system, including:

- Security
- Performance
- Reliability
- External interfaces

Also commonly referred to as end-to-end testing.

Acceptance Testing

Acceptance testing is the process of verifying that desired acceptance criteria are met from the user's perspective and that the system satisfies the business requirements.

Each use-case should be covered by one or more acceptance tests.

While this is a more objective view of a system, it can still be automated.

Black-box/White-box Testing

Black-box testing:

- No programming/software/implementation knowledge.
- Tests based only on specification or design.
- Usually used for System and Acceptance testing.
- An external tester is ideal as they won't have any bias based on knowledge of the implementation.

White-box testing:

- Utilize knowledge of the implementation to design tests cases to cover key paths.
- Usually used for Unit and Integration testing.
- Written by the developer to verify their code.

Regression Testing

Regression testing is the process of verifying that incremental changes have not broken existing functionality.

It makes sense to run the full suites of tests after any code, infrastructure or configuration change. These tests can almost immediately identify issues and help narrow down any problems.

This helps to highlight possible errors and/or backwards incompatible changes when run preemptively during development.

A comprehensive suite can take minutes, need to determine where the best place to run these. A common approach is after any manual or unit tests are run as another check and again before commit/push.

Brute-forcing all input combinations?

Can we just brute force all possible inputs and test them all?

- Yes, sometimes. But it's not a great idea and almost never worth the time.

Most systems are complex enough that the sheer number of combinations of inputs means you'll be running tests for a long period of time when attempting them all.

- Consider a phone:
 - Does it make sense to attempt to call every single phone number to verify call functionality? Is that a good use of time if it took weeks?
- There will still be gaps:
 - Concurrency
 - Order of inputs or timing of requests
 - Random seeds/input
 - Hardware failure

Fuzzing

Automated tools that can generate invalid and non-standard inputs and test the behavior of a system

The idea with fuzzing is to define the valid inputs and have the fuzzer generate input that is semi-valid to test boundary conditions.

They can highlight:

- Unhandled edge cases
- Non-performant code
- Possible crashes (memory leaks) and security concerns

Non-functional

Performance and Load Testing

Determines how well a system performs under known workloads. By adjusting the stress factor, we can simulate high load and identify bottlenecks in our system.

Volume testing is a way to test functionality under load.

Stress testing is a way to test reliability under load.

Load testing is a way to test performance under load.

Important to ensure that simulated workload is realistic.

We want the site to handle 10,000 concurrent users during peak so we'll test how it performs with: 1000, 5000, 10,000, 15,000 simulated users.

Security Testing

Required to ensure that our system is secure.

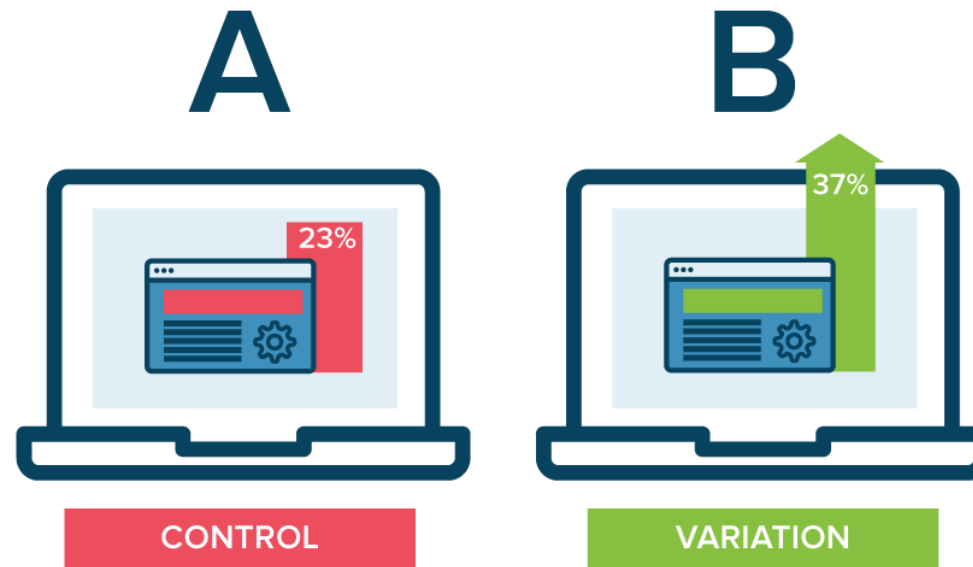
The top 10 vulnerability list?

Usability Testing

Designed to test whether our website is intuitive and discoverable by users.

Some common techniques:

- Hallway testing
- Expert review
- A/B testing



Robustness/Destructive Testing

Destructive testing intentionally causes failures in components of the system to test how the system performs.

Designed to answer the following:

- Will our system recover if it fails?
- Will we lose data?
- How long will it take to recover?
- Do we have backups?
- Do they work?
- Who knows how to fix?

Netflix Simian Army/Chaos Monkey



Netflix Simian Army

Netflix desires high uptime and availability and they have developed a suite of automated tools to test how well their production systems handle particular failure/abnormal scenarios:

- Chaos Monkey: Randomly disables production instances.
- Latency Monkey: Introduces artificial delay in the client-server communication layer.
- Conformity Monkey: Detects abnormal instances that don't adhere to best practices and terminates them.
- Doctor Monkey: Detects instances that are unhealthy and routes traffic away from them. This allows for time to investigate without sacrificing system stability.
- Chaos Gorilla: Big Chaos Monkey. Simulates outage of an entire availability zone.

Testing Approaches

Manual vs Automated Testing

Manual testing is directly performed by a human operator, whether that be the developer or a dedicated QA agent.

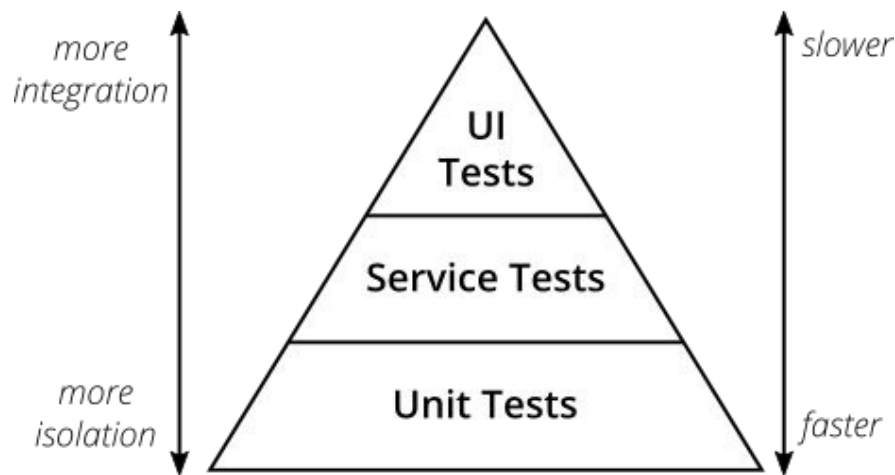
- Exploratory testing
- Visual regression testing

Automated test are comprised of many different tools that are driven by a computer.

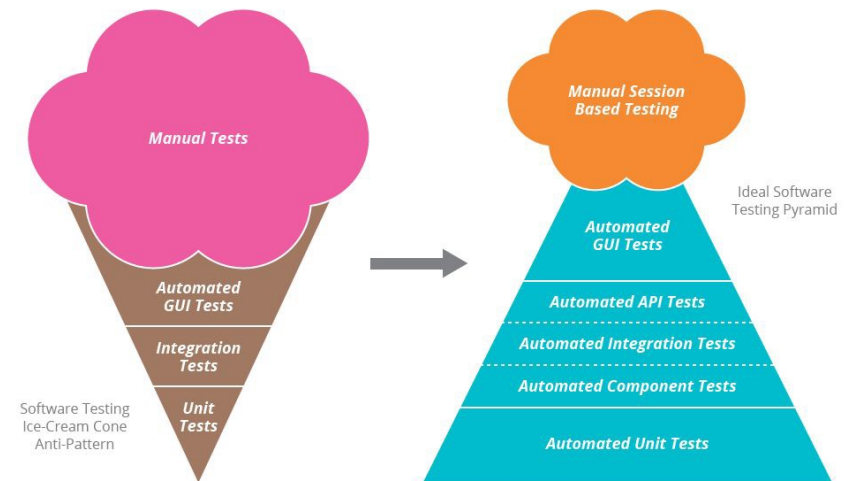
- End-to-end (E2E) tests
- Unit tests
- Integration tests
- Performance tests

Which Testing Approach?

The Test Pyramid is a great way of conceptualizing where the majority of your tests should be. It minimizes the amount of time spent on tests for the most value. Leads to issues being caught earlier in development.



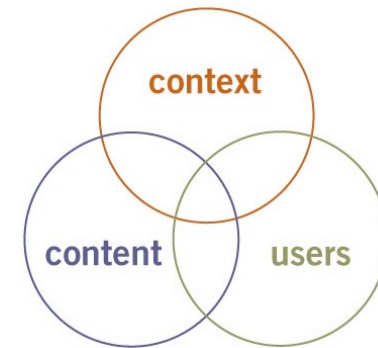
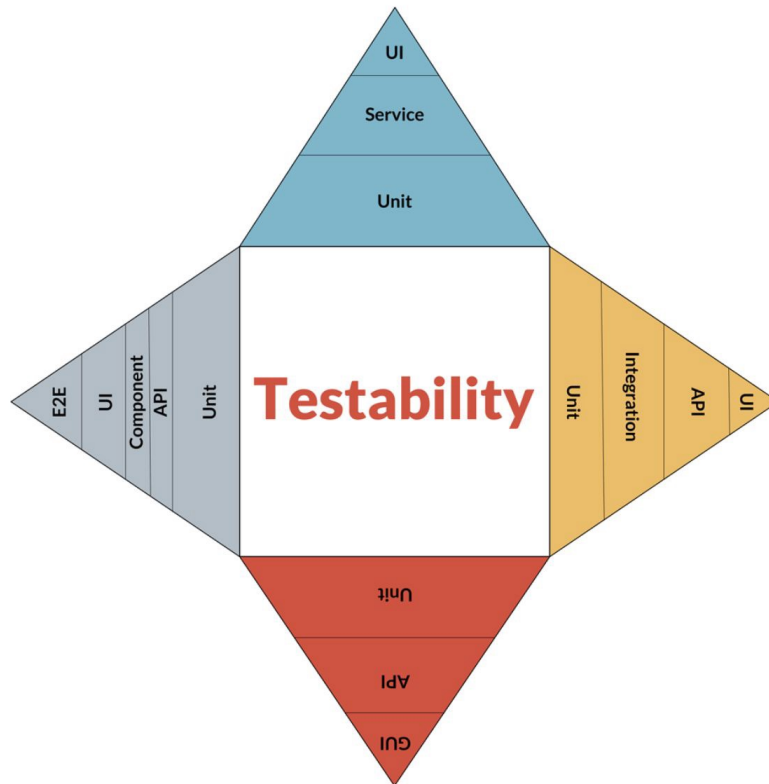
Practical Test Pyramid



Ice Cream Anti-Pattern

Which Testing Approach?

Perhaps the 3D Testability Polygon is more your style.



Which Testing Approach?

- It's more important to cover your acceptance criteria than to adhere rigorously to a testing strategy. Find an approach that works for your framework, the nature of your program and requirements of your users.
- Consider the cost/value of each test you write.
- Consider the cost of issues x likelihood for each test case you don't.

Linting

What is linting?

Linters are tools which run over your program and identify programmatic and stylistic errors.

- Unused imports
- Unreachable code
- Variable/class/function names that don't conform to style guidelines

Designing Test Cases

How to design good test cases?

- Prefer **quality** of tests over **quantity**.
- Diversity is key.
- Aim to cover as many unique tests cases as possible.
- Cover the basic test cases before looking for the more esoteric edge cases.
- Always consider what could happen if someone was intentionally malicious.
- Don't forget about possible bugs in libraries. Would your code handle it?

Example 1: Odd/Even

- Consider the following Python function:

```
def oddEven(num):  
    # Number is divisible by 2 so is even  
    if num % 2 == 0:  
        return "Even"  
    # Number must be odd  
    return "Odd"
```

- Does it work? How would we know?

Example 1: Odd/Even

Test cases:

- Odd numbers: 1, 2, 3
- Even numbers: 0, 2, 4, 6
- Negative numbers: -1, -2, -3
- Large numbers: Max int
- Invalid data types: String, Boolean, Float
- None/Null/Undefined

Equivalence Partitioning

When making tests cases, it is important to identify clusters of inputs that are functionally equivalent or treated similarly by the code.

In the previous example, all odd numbers were treated equally by the function, as were all even. When generating tests, there is reduced value in multiple odd number tests as we are not covering new failure cases. In this case we are partitioning inputs based on even-ness.

Boundary cases are a common cause for issues as we'll see in the next example. Does **oddEven** handle 0 correctly?

Example 2: Palindromes

Consider the function *isPalindrome(word)* which takes a word and returns if it is a palindrome or not.

What test cases should we have?

- “madam”
- “12321”
- “Madam, in Eden, I’m Adam”

Example 2: Palindromes

We should consider the following cases:

- Even length words
- Odd length words
- Empty string: ""
- Invalid data type: Number, Object, Boolean...
- None/Null/Undefined
- Unicode. Multi-byte characters
- Right-to-left language and/or multi-byte characters

Guideline Based Testing

A lot of code is vulnerable to common logic flaws made by programmers. Buffer overflows in common programs/libraries are a great example.

Focus more on issues that are likely to slip by a developer and draw their attention to it.

Did you consider Unicode when handling strings?

Did you free that memory you allocated?

Static analysis tools can help automate this technique.

Example 3 – User Signup Form

First name:

Last name:

Date of Birth:

Email:

What test cases should we have for our signup-form?

Example 3 – User Signup Form

We should consider the following cases:

- Empty strings: first/last name, email and date of birth
- Missing parameters from the request
- Long string fields: names/email
- Unicode string fields: names/email
- Invalid dates: 3000, -1700, 40/15
- Numeric characters in first/last name
- Emails should match some format

Code Coverage

Code Coverage

It is good to estimate which parts of your code are covered by tests.

Code coverage tools can highlight, line-by-line, which parts of code are executed by tests. However, just because code has 100% coverage, does not mean that it is bug-free - just that there is at least one test that executes this statement.

Would we be satisfied with only one test for any of our example methods covered in previous slides?

Code coverage is only an indicator (not a direct measure) of quality.

Code Coverage

Function Coverage: how many of the functions defined have been called?

Statement Coverage: how many of the statements have been executed?

Branch Coverage: how many of the branches of control flow structures have been executed? if/elif/else, while, for, switch

Condition Coverage: how many of the boolean conditions within branches evaluated to both true and false?

Line Coverage: raw metric of how many lines have been tested.

Writing Defensive Code

Defensive Programming

- Hand-in-hand with verifying the quality of our web application, we can eliminate entire classes of bugs through good code design.
- Defensive programming comes at the cost of more complicated code, but will ensure that your application can deliver on availability and security requirements.

Defensive Programming

- Consider the two programs below that open a file and display its output.

```
for line in open("file.txt"):
    print(line)
```

vs

```
try:
    with open("file.txt") as f:
        for line in f.readlines():
            print(line)
except (IOError) as e:
    # handle exception
except (Exception) as e:
    # handle exception
```

- Always take advantage of language constructs (in-built design patterns).

Design Patterns

Design patterns exist for a reason. They represent consensus from seasoned developers that these patterns will solve common problems.

**You reached the end of the lecture.
Thank you and see you next week.**